

Tasks: Tokyo Mate 東京通

Input: `/specs/001-tokyo-travel-assistant/` 下的設計文件

Prerequisites: `plan.md`、`spec.md`、`research.md`、`data-model.md`、`contracts/`、`quickstart.md`、`ux-ui-design-handoff.md`

Tests: 規格要求核心行為、失敗狀態與 Runnable Validation Journeys 可驗證，因此各 User Story 均包含先寫且先失敗的 Vitest / React Testing Library 測試。

Organization: 任務依 User Story 分組，使每個故事都能獨立實作、驗證與交付。

Tasks Phase 與 Technical Plan Phase 對照

Tasks Phase 與 Technical Plan Phase 是不同維度；數字不代表一一對應。下表提供每個 Tasks Phase 的追溯關係，不為了數字一致而重新切割 User Story tasks。

Tasks Phase	對應 Technical Plan Phase
Phase 1 Setup	Phase 1 App Shell & Foundation
Phase 2 Foundational	Phase 1 App Shell & Foundation、Phase 2 Server Boundary & 文字 AI 核心
Phase 3 User Story 1	Phase 3 中日雙向翻譯體驗、Phase 6 Speech-to-Text & Text-to-Speech
Phase 4 User Story 2	Phase 4 Tokyo Knowledge Base & 檢索
Phase 5 User Story 3	Phase 5 即時 Web Data 狀態處理、Phase 7 Explore Nearby、Phase 8 Emergency Mode & Graceful Error Recovery
Phase 6 Polish & Cross-Cutting Concerns	Phase 1 App Shell & Foundation、Phase 9 Responsive Polish & Acceptance Criteria Verification

Requirement Traceability Matrix

本表為正式 Requirement Traceability Matrix，逐一手動對應目前已存在的 Task ID 與 Verification / artifact，不依賴分析器自行 inference；涵蓋 spec.md 全部 FR-001 ~ FR-026 與 SC-001 ~ SC-014，並額外列出 Session UI State Contract 之追溯關係。

Requirement ID	對應 Task ID(s)	Verification / Artifact
FR-001	T025, T029, T030	<code>api/assistant.translation.test.ts</code> 自動語言方向 contract tests； <code>quickstart Journey 1</code> ； <code>docs/verification/journey-report.md</code>
FR-002	T025, T029, T030	<code>api/assistant.translation.test.ts</code> 情境化自然翻譯 contract tests； <code>quickstart Journey 1</code>
FR-003	T025, T029, T030	<code>api/assistant.translation.test.ts</code> 中日雙向 contract tests； <code>quickstart Journey 1</code>
FR-004	T026, T028, T031, T033, T034, T035, T036	<code>api/transcribe.test.ts</code> 、 <code>src/features/translation/TranslationJourney.test.tsx</code> ； <code>quickstart Journey 1</code>

Requirement ID	對應 Task ID(s)	Verification / Artifact
FR-005	T027, T032, T036, T037	api/speech.test.ts ; quickstart Journey 1 (正常 / 慢速播放、語調切換)
FR-006	T028, T037	src/features/translation/TranslationJourney.test.tsx ; quickstart Journey 1 (複製與高可讀性展示)
FR-007	T028, T034	src/features/translation/TranslationJourney.test.tsx (麥克風拒絕流程) ; quickstart Journey 1
FR-008	T042, T050, T051, T052	api/assistant.travel.test.ts ; src/services/knowledge.test.ts ; quickstart Journey 2
FR-009	T042, T043, T051, T052, T053	api/assistant.travel.test.ts 、 src/features/knowledge/KnowledgeJourney.test.tsx ; SC-003 驗收於 T087 項目(3)
FR-010	T041, T044, T045, T046, T047, T048, T049, T050	src/services/knowledge.test.ts ; quickstart Journey 2 (六類知識瀏覽)
FR-011	T039, T040	quickstart Journey 0 / 1 (首頁共用輸入入口)
FR-012	T058, T063, T064, T067	api/places.test.ts 、 src/services/geolocation.test.ts ; quickstart Journey 3
FR-013	T024a, T059, T063	src/test/platform-safety-gate.test.ts 、 src/services/geolocation.test.ts ; quickstart Journey 0 / 3
FR-014	T059, T063, T067	src/services/geolocation.test.ts ; quickstart Journey 3 (拒絕後手動地區 fallback)
FR-015	T060, T068, T069, T071, T072	api/assistant.live-data.test.ts ; quickstart Journey 4 ; FR-015 驗收於 T087 項目(5) ; docs/verification/journey-report.md
FR-016	T061, T068, T070, T071, T073	api/assistant.emergency.test.ts ; quickstart Journey 5 ; SC-004 驗收於 T087 項目(4)

Requirement ID	對應 Task ID(s)	Verification / Artifact
FR-017	T021, T024a, T084	src/test/platform-safety-gate.test.ts 、 src/service-worker.test.ts ; FR-017/SC-009 驗收於 T087 項目(6) ; docs/verification/privacy-report.md
FR-018	T024a, T063, T084	src/services/geolocation.test.ts (座標不持久化)、 src/service-worker.test.ts ; docs/verification/privacy-report.md
FR-019	T025, T029, T030	api/assistant.translation.test.ts (Tone QA fixture : 餐廳 / 飯店 / 購物 / 交通各 ≥2 案例) ; SC-008 驗收 ; docs/verification/journey-report.md
FR-020	T025	api/assistant.translation.test.ts (台灣繁中詞彙 QA : 計程車/飯店/行動電源/便利商店/網路) ; FR-020 驗收於 T087 項目(7)
FR-021	T075, T079, T080	quickstart 安裝流程驗證 ; docs/verification/pwa-report.md
FR-022	T077, T081	quickstart 離線 App Shell 驗證 ; docs/verification/pwa-report.md
FR-023	T078, T082	src/components/NetworkStatus.test.tsx ; docs/verification/pwa-report.md
FR-024	T077, T083	src/components/UpdatePrompt.tsx 驗證 ; docs/verification/pwa-report.md
FR-025	T085, T089	quickstart Journey 0 viewport matrix ; docs/verification/pwa-report.md
FR-026	T078, T084	src/service-worker.test.ts ; docs/verification/privacy-report.md
SC-001	T040, T087	quickstart Journey 1 (四步驟操作) ; T087 項目(1) ; docs/verification/journey-report.md
SC-002	T087	tests/fixtures/latency-cases.json (30+ 案例) ; docs/verification/latency-report.md
SC-003	T042, T043, T053, T087	src/features/knowledge/KnowledgeJourney.test.tsx ; T087 項目(3) ; docs/verification/journey-report.md
SC-004	T061, T073, T087	api/assistant.emergency.test.ts ; quickstart Journey 5 ; T087 項目(4)
SC-005	T059, T063, T067	src/services/geolocation.test.ts ; quickstart Journey 3
SC-006	T060, T072, T087	api/assistant.live-data.test.ts ; quickstart Journey 4 ; T087 項目(5)
SC-007	T024a, T039, T063	src/test/platform-safety-gate.test.ts ; quickstart Journey 0

Requirement ID	對應 Task ID(s)	Verification / Artifact
SC-008	T025, T087	<code>api/assistant.translation.test.ts</code> (Tone QA fixture) ; T087 項目(7) ; <code>docs/verification/journey-report.md</code>
SC-009	T021, T084, T087	<code>src/service-worker.test.ts</code> ; T087 項目(6) ; <code>docs/verification/privacy-report.md</code>
SC-010	T075, T079, T080	quickstart 安裝流程驗證 ; <code>docs/verification/pwa-report.md</code>
SC-011	T077, T081	quickstart 離線 App Shell 驗證 ; <code>docs/verification/pwa-report.md</code>
SC-012	T078, T082	<code>src/components/NetworkStatus.test.tsx</code> ; <code>docs/verification/pwa-report.md</code>
SC-013	T077, T083	<code>src/components/UpdatePrompt.tsx</code> 驗證 ; <code>docs/verification/pwa-report.md</code>
SC-014	T085, T089, T087	quickstart Journey 0 viewport matrix ; T087 項目(8) ; <code>docs/verification/pwa-report.md</code>
Session UI State Contract (spec.md)	T021, T023, T024, T024a, T034, T035, T038, T063, T084	<code>src/test/platform-safety-gate.test.ts</code> 、 <code>src/service-worker.test.ts</code> ; quickstart 全 Journey 手動檢查 ; <code>docs/verification/journey-report.md</code>

Generated Verification Artifacts

以下 verification artifacts 由對應 Verify 任務執行時產生；尚未執行對應任務前，不要求檔案預先存在。

Artifact	Owner Task(s)	Trigger	Output Path	Minimum Contents
Latency Report	T087	執行 SC-002 標準化 latency 量測 (以 <code>tests/fixtures/latency-cases.json</code> 進行 30+ 次代表性互動測試)	<code>docs/verification/latency-report.md</code>	date 、 environment 、 commit 、 case id 、 category 、 latency (ms) 、 pass/fail 、 p50 、 p95 、 maximum 、 failed/outlier cases
Privacy Report	T084	執行 FR-017 / FR-018 / FR-026 / SC-009 之 cache 與 storage privacy boundary 驗證	<code>docs/verification/privacy-report.md</code>	date 、 environment 、 commit 、 檢查之 storage (LocalStorage/IndexedDB/Cache Storage) 、 逐項檢查結果 (對話內容、翻譯結果、語音 blob 、 transcription 、 TTS audio 、 精確位置、Places 個人情境結果、即時 API response 、 credentials/secrets) 、 pass/fail 、 deferred issues

Artifact	Owner Task(s)	Trigger	Output Path	Minimum Contents
PWA Report	T080, T081, T082, T083, T085, T089	執行 FR-021 ~ FR-025 / SC-010 ~ SC-014 之 installability、offline App Shell、network-required fallback、update flow 與 viewport matrix 驗證	docs/verification/pwa-report.md	date、environment、commit、installability 結果、offline reload 結果、network-required fallback 結果、update prompt 行為、viewport matrix (360×800 / 390×844 / 430×932 / 844×390) 逐項 safe-area / 鍵盤 / 水平捲動 / touch target / standalone 結果、pass/fail、deferred issues
Journey Report	T087, T089	執行 Journey 0-5 手動驗收 (含 SC-001、SC-003、SC-004、SC-006、SC-008、FR-015、FR-016、FR-019、FR-020)	docs/verification/journey-report.md	date、environment、commit、各 Journey 操作步驟與觀察結果、Tone QA fixture (餐廳 / 飯店 / 購物 / 交通各 ≥2 案例) 結果、台灣繁中詞彙 QA fixture 結果、pass/fail、deferred issues
Final Report	T088	完成 T001-T089 全部 Verify 並彙整上述四份 report 後執行最終彙整	docs/verification/final-report.md	date、environment、commit、Requirement Traceability coverage 總結 (FR-001 ~ 026、SC-001 ~ 014 是否全數覆蓋)、四份子 report pass/fail 彙總、Constitution Check 狀態、CRITICAL/HIGH findings (如有)、stack drift 檢查結果、發布建議

Format: [ID] [P?] [Story] Description

- **[P]:** 可平行執行，因為使用不同檔案且不依賴尚未完成的任務
- **[Story]:** 對應 spec.md 的 User Story (US1、US2、US3)
- 每項任務均包含明確檔案路徑

Path Conventions

- React 前端位於 **src/**
- Vercel Serverless Functions 位於 **api/**
- 靜態與 PWA 資產位於 **public/**
- 測試與被測程式相鄰，使用 *.test.ts 或 *.test.tsx

Phase 1: Setup (共用基礎設施)

Technical Plan Traceability: Phase 1 App Shell & Foundation

Purpose: 建立 React、TypeScript、Vite、Vitest 與 Vercel 的非破壞性專案骨架。

- ☒ T001 建立 React 18、TypeScript 5、Vite、Vitest、React Testing Library 與 OpenAI SDK 相依套件及 scripts 於 package.json
- ☒ T002 [P] 設定嚴格 TypeScript 編譯、DOM 型別與 JSON module 支援於 tsconfig.json
- ☒ T003 [P] 設定 React Vite 建置與 /api 開發代理於 vite.config.ts
- ☒ T004 [P] 設定 jsdom、測試 setup 與 coverage 規則於 vitest.config.ts 及 src/test/setup.ts
- ☒ T005 [P] 建立 OpenAI、Google Places 與模型名稱的無秘密範本於 .env.example
- ☒ T006 [P] 設定 Vercel SPA rewrite、Function runtime 與安全回應標頭於 vercel.json
- ☒ T007 建立 Vite HTML 入口、PWA metadata 與主題色於 index.html

Run: 啟動目前版本的安裝與基礎開發環境，執行 **npm install**。

Verify: 執行 **npm run build** 與空測試套件，確認既有 .git/、.github/、.specify/、.vscode/、specs/ 不被覆寫。

Fix: 若 Verify 失敗，只修正 Phase 1 直接相關的設定或基礎檔案，然後重新 Run / Verify。

Evidence: 記錄 commands、通過的 build/test checks、未通過項目與 deferred issues；不要求每個小 task 建立獨立報告。

Phase 2: Foundational (阻塞所有故事的共用能力)

Technical Plan Traceability: Phase 1 App Shell & Foundation、Phase 2 Server Boundary & 文字 AI 核心

Purpose: 建立所有 User Story 共用的資料契約、server boundary、App Shell、錯誤處理、網路狀態與早期平台安全檢核閘門。

⚠ **CRITICAL GATE: Early Platform Safety Gate:** 此階段與 Early Platform Safety Gate 完成並驗證通過前，嚴禁開始任何 User Story 實作。

- ☑ T008 [P] 定義 UserRequest、UserLocation、InputType 與 UserTone 型別及邊界規則於 src/types/request.ts
- ☑ T009 [P] 依 Specification FR-015 / FR-016 與 data-model.md 之 canonical 決策維度 / AssistantResult 契約，定義對應 TypeScript 型別於 src/types/assistant.ts
- ☑ T010 [P] 定義 KnowledgeEntry 與 KnowledgeCategory 型別於 src/types/knowledge.ts
- ☑ T011 [P] 定義 PlaceResult 與附近搜尋條件型別於 src/types/place.ts
- ☑ T012 [P] 定義錄音轉錄與語音合成 request/result 型別於 src/types/speech.ts
- ☑ T013 [P] 定義 ProductError、ErrorCode 與使用者可行動錯誤文案映射於 src/types/error.ts
- ☑ T014 建立 API 成功 / 失敗 envelope、method guard、JSON body 與欄位驗證 helper 於 api/_lib/http.ts
- ☑ T015 [P] 建立僅於 server 讀取 OPENAI_API_KEY、OPENAI_MODEL 的 OpenAI client factory 於 api/_lib/openai.ts
- ☑ T016 建立 assistant 結構化輸出 schema (支援獨立 Safety/Freshness 與最小契約)、provider response mapping 與共用 system rules 於 api/_lib/assistant-schema.ts
- ☑ T017 建立 assistant、transcribe、speech、places 的型別化 fetch wrapper 與 ProductError 轉換於 src/services/api.ts
- ☑ T018 [P] 建立可重用 Button、IconButton、Input、StatusMessage 與 Modal 元件於 src/components/
- ☑ T019 [P] 建立色彩、字級、間距、觸控尺寸與 safe-area tokens 於 src/styles/variables.css
- ☑ T020 建立 mobile-first reset、焦點樣式與基礎版面於 src/styles/global.css
- ☑ T021 建立首頁 / 助手 / 附近 / 百科的單一 App Shell 狀態，並確保 user interaction state 僅存在 browser runtime memory (React state / Context) 而不寫入 LocalStorage/IndexedDB 於 src/App.tsx
- ☑ T022 建立 React 掛載點與全域樣式匯入於 src/main.tsx
- ☑ T023 [P] 建立 online/offline 監聽、需要連線能力判斷與重試狀態於 src/services/network.ts
- ☑ T024 建立連線狀態提示與不捏造成果結果的離線邊界於 src/components/NetworkStatus.tsx
- ☑ T024a 驗證 Early Platform Safety Gate：確認 service worker 僅在 implementation 存在後才註冊、/api/* 與即時/AI/位置 requests 排除於 cache 外、AI response / 語音 blob / 精確位置不持久化、LocalStorage/IndexedDB 不存對話、manifest 基本有效、Cache allowlist 僅限 approved App Shell/assets/Knowledge Base 於 src/test/platform-safety-gate.test.ts

Run: 啟動目前版本的 App Shell、server boundary、安全閘門測試與測試環境。

Verify: 執行 Foundational automated tests、Early Platform Safety Gate 測試、TypeScript/static checks 與 manual checks，確認共用型別不暴露 provider raw response、首頁不要求位置或麥克風權限、外部 API 失敗可轉換為繁體中文 ProductError、完全符合 Early Platform Safety Gate 條件。

Fix: 若 Verify 失敗，只修正 Phase 2 直接相關的型別、boundary、App Shell 或錯誤處理檔案，然後重新 Run / Verify；Gate 未通過前不得進入 Phase 3。

Evidence: 記錄 commands、automated checks、manual journeys、pass/fail 與 deferred issues；不要求每個小 task 建立獨立報告。

Phase 3: User Story 1 - 即時旅遊翻譯與口譯 (Priority: P1) 🚀 MVP

Technical Plan Traceability: Phase 3 中日雙向翻譯體驗、Phase 6 Speech-to-Text & Text-to-Speech

Goal: 讓旅客以中文或日文文字 / 語音完成自動辨向、自然翻譯、語調調整、複製及正常 / 慢速播放。

Independent Test: 從首頁輸入或錄製餐廳情境中文，修正辨識文字後取得自然日文；切換禮貌語氣、複製並播放正常與 0.75x 慢速語音。拒絕麥克風後仍可完整使用文字流程。

Tests for User Story 1

先寫測試並確認失敗，再開始對應實作。

- ☑ T025 [P] [US1] 依 Specification FR-019 / FR-020 與 Plan 的 QA implementation 執行對應驗證，撰寫自動語言方向、台灣繁體詞彙與 Tone QA fixture 之 Assistant contract tests 於 api/assistant.translation.test.ts
- ☑ T026 [P] [US1] 撰寫音訊存在、10 MB、MIME 白名單與錯誤 envelope 的 Transcribe contract tests 於 api/transcribe.test.ts
- ☑ T027 [P] [US1] 撰寫文字長度、語言、正常 / 慢速速度與 audio response 的 Speech contract tests 於 api/speech.test.ts

- ☒ T028 [P] [US1] 撰寫錄音、可編輯 transcript、語調、複製、播放及麥克風拒絕旅程測試於 `src/features/translation/TranslationJourney.test.tsx`

Implementation for User Story 1

- ☒ T029 [P] [US1] 建立自動辨識中日文、情境化自然翻譯與 `default/polite/casual` 規則於 `api/_lib/prompts/translation.ts`
- ☒ T030 [US1] 實作 POST `/api/assistant` 的 request validation、Structured Output 呼叫與 translation 結果 mapping 於 `api/assistant.ts`
- ☒ T031 [P] [US1] 實作 multipart 音訊檢查、10 MB/MIME 限制、Whisper 轉錄與友善錯誤於 `api/transcribe.ts`
- ☒ T032 [P] [US1] 實作文字 / 語言 / 速度檢查、OpenAI Speech 產生與 audio response 於 `api/speech.ts`
- ☒ T033 [P] [US1] 實作跨瀏覽器 MIME 選擇、MediaRecorder lifecycle 與錄音 Blob 釋放於 `src/services/recorder.ts`
- ☒ T034 [P] [US1] 建立錄音中、停止、轉錄中、拒絕與改用文字狀態於 `src/features/speech/VoiceInputModal.tsx`
- ☒ T035 [P] [US1] 建立辨識文字可修改、確認送出與保留原輸入的介面於 `src/features/speech/EditableTranscript.tsx`
- ☒ T036 [P] [US1] 建立可取消前次 object URL、正常 / 慢速載入與播放失敗 fallback 的播放器於 `src/components/AudioPlayer.tsx`
- ☒ T037 [P] [US1] 建立大字目標語言、發音、複製、播放及語調 controls 於 `src/features/translation/TranslationResult.tsx`
- ☒ T038 [US1] 建立 assistant loading/error/result 狀態與語調重送流程於 `src/screens/AssistantScreen.tsx`
- ☒ T039 [US1] 建立品牌、共用文字輸入、麥克風及四個入口的首頁於 `src/screens/HomeScreen.tsx`
- ☒ T040 [US1] 串接 HomeScreen、VoiceInputModal、AssistantScreen 與 TranslationResult 的完整本次互動流程於 `src/App.tsx`

Run：啟動目前版本的翻譯、語音與口譯流程環境。

Verify：執行 US1 automated tests、build/type checks 與 Journey 1 manual check，確認雙向翻譯、修正、語調、複製、語音播放與麥克風拒絕後的文字 fallback。

Fix：若 Verify 失敗，只修正 Phase 3 直接相關的 source/test/config，然後重新 Run / Verify。

Evidence：記錄 commands、通過的 journey/checks、未通過項目與 deferred issues；不要求每個小 task 建立獨立報告。

Phase 4: User Story 2 - 東京旅遊知識與行動建議 (Priority: P1)

Technical Plan Traceability: Phase 4 Tokyo Knowledge Base & 檢索

Goal: 讓旅客按地點、時間、同行者、偏好與限制取得 action-first 東京建議，並可瀏覽六類穩定知識。

Independent Test: 詢問淺草 / 上野半日安排、新宿轉乘與素食餐飲，確認回答依「最推薦 → 怎麼做 → 注意 → 必要日文」呈現，並可離線瀏覽對應百科內容。

Tests for User Story 2

先寫測試並確認失敗，再開始對應實作。

- ☐ T041 [P] [US2] 撰寫區域、分類、標籤與意圖加權後選出 Top 3-5 筆的知識檢索測試於 `src/services/knowledge.test.ts`
- ☐ T042 [P] [US2] 撰寫地點、時間、同行者、偏好、限制與 SC-003 一般旅遊最小契約 (結論 conclusion、行動 action、可選注意 caution、可選日文 phrase) 的 Assistant contract tests，涵蓋區域選擇、交通、美食與兩天活動代表性案例於 `api/assistant.travel.test.ts`
- ☐ T043 [P] [US2] 撰寫百科分類、主要內容區塊順序 (結論 → 做法 → 注意 → 必要日文)、無需展開即可看到結論與第一個行動，以及旅遊回答 contextual actions 的元件測試於 `src/features/knowledge/KnowledgeJourney.test.tsx`

Implementation for User Story 2

- ☐ T044 [P] [US2] 建立至少 18 個指定東京區域的結構化指南於 `src/data/tokyo/areas.json`
- ☐ T045 [P] [US2] 建立 JR、Metro、都營、私鐵、IC 卡、機場與主要車站知識於 `src/data/tokyo/transport.json`
- ☐ T046 [P] [US2] 建立餐飲類型、點餐、排隊、預約、付款、過敏與素食知識於 `src/data/tokyo/food.json`
- ☐ T047 [P] [US2] 建立藥妝、百貨、電器、品牌、動漫、二手、超市與免稅知識於 `src/data/tokyo/shopping.json`
- ☐ T048 [P] [US2] 建立電車、排隊、垃圾、溫泉、宗教場所、餐廳與公共場所禮儀於 `src/data/tokyo/culture.json`
- ☐ T049 [P] [US2] 建立疾病、護照 / 手機 / 行李遺失、警察、醫療與災害穩定知識於 `src/data/tokyo/emergency.json`
- ☐ T050 [US2] 實作六類 JSON 載入、metadata scoring、Top 3-5 選取與 context 序列化於 `src/services/knowledge.ts`
- ☐ T051 [P] [US2] 建立旅遊問題 context extraction 與「結論 conclusion、行動 action、可選注意 caution、可選實用日文 phrase (僅在對行動有幫助時提供)」prompt 規則於 `api/_lib/prompts/travel.ts`
- ☐ T052 [US2] 將知識 context 注入、travel/knowledge intent 與 action_plan / knowledge_summary mapping 整合至 `api/assistant.ts`

- ☐ T053 [P] [US2] 建立依序呈現結論、步驟、注意事項與可選實用日文的旅遊回答版型於 `src/features/travel/TravelAnswer.tsx`
- ☐ T054 [P] [US2] 建立一致順序呈現區域 / 主題內容的知識條目元件於 `src/features/knowledge/KnowledgeCard.tsx`
- ☐ T055 [P] [US2] 建立六分類 segmented control 與無結果狀態於 `src/features/knowledge/CategoryFilter.tsx`
- ☐ T056 [US2] 建立可離線讀取、篩選與展開知識條目的百科畫面 (KnowledgeBrowser 僅為東京百科 UI component 識別名稱，見 `spec.md Naming Glossary`) 於 `src/screens/KnowledgeBrowser.tsx`
- ☐ T057 [US2] 串接首頁「問東京 / 東京百科」、AssistantScreen 旅遊回答與本次暫存狀態於 `src/App.tsx`

Run : 啟動目前版本的旅遊問答、知識庫與離線靜態內容環境。

Verify : 執行 US2 automated tests、build/type checks 與 Journey 2 manual checks，並以 SC-003 的區域選擇、交通、美食、兩天活動案例驗證回答順序；確認無網路時可瀏覽六類靜態知識。

Fix : 若 Verify 失敗，只修正 Phase 4 直接相關的 source/test/config，然後重新 Run / Verify。

Evidence : 記錄 commands、automated checks、manual journeys、pass/fail 與 deferred issues；不要求每個小 task 建立獨立報告。

Phase 5: User Story 3 - 東京區域探索、即時資訊與緊急處理 (Priority: P2)

Technical Plan Traceability: Phase 5 即時 Web Data 狀態處理、Phase 7 Explore Nearby、Phase 8 Emergency Mode & Graceful Error Recovery

Goal: 讓旅客主動授權定位或手動輸入地區探索附近，以獨立雙維度 (Safety / Freshness) 辨識動態資訊可信度，並在緊急情境優先取得安全行動與日文。

Independent Test: 首次載入不出現定位權限；可用 GPS 或拒絕後以「淺草」搜尋附近地點；詢問營業時間可見 verified/uncertain 狀態；輸入護照遺失時只顯示安全優先指引。

Tests for User Story 3

先寫測試並確認失敗，再開始對應實作。

- ☐ T058 [P] [US3] 撰寫座標 / 手動地區、分類、Google response mapping 與 Places 錯誤 contract tests 於 `api/places.test.ts`
- ☐ T059 [P] [US3] 撰寫僅由使用者動作請求定位、拒絕 fallback 與座標不持久化測試於 `src/services/geolocation.test.ts`
- ☐ T060 [P] [US3] 依 Specification FR-015 與 Plan 對應 technical contract，撰寫 Freshness 決策維度、即時訊息與 nextAction 之 Assistant contract tests 於 `api/assistant.live-data.test.ts`
- ☐ T061 [P] [US3] 依 Specification FR-016 與 Plan 對應 technical contract，撰寫 Safety 決策維度、緊急指引最小契約與隱藏無關旅遊推薦之 Assistant contract tests 於 `api/assistant.emergency.test.ts`
- ☐ T062 [P] [US3] 撰寫 GPS、手動地區、地點卡、狀態標籤與緊急卡畫面旅程測試於 `src/features/nearby/NearbyEmergencyJourney.test.tsx`

Implementation for User Story 3

- ☐ T063 [P] [US3] 實作按使用者動作取得座標、權限拒絕 / 不可用 mapping 且不持久化位置於 `src/services/geolocation.ts`
- ☐ T064 [US3] 實作 Places request validation、Nearby/Text Search、欄位遮罩、距離計算與友善錯誤於 `api/places.ts`
- ☐ T065 [P] [US3] 建立地點名稱、推薦理由、距離、保守營業狀態與「怎麼去」操作於 `src/features/nearby/PlaceCard.tsx`
- ☐ T066 [P] [US3] 建立載入、結果、空結果與 Places 不可用狀態列表於 `src/features/nearby/NearbyResults.tsx`
- ☐ T067 [US3] 建立「使用我的位置 / 輸入地區」、分類與定位拒絕 fallback 的附近探索畫面於 `src/features/nearby/NearbyExplorer.tsx`
- ☐ T068 [P] [US3] 依 Specification FR-015 / FR-016 與 Plan 「Emergency + Live-Required 兩階段技術機制」，建立 web search 呼叫、來源時間、Freshness 正規化，並支援可設定 bounded timeout (emergency + live_required Stage 1 使用短逾時、Stage 2 補充 enrichment 使用完整逾時) 於 `api/_lib/live-search.ts`
- ☐ T069 [P] [US3] 依 Specification FR-015 與 Plan 對應 technical contract，建立 live-data 判斷與 nextAction 後續步驟 prompt 規則於 `api/_lib/prompts/live-data.ts`
- ☐ T070 [P] [US3] 依 Specification FR-016 與 Plan 對應 technical contract，建立緊急意圖判斷與安全優先順序 prompt 規則於 `api/_lib/prompts/emergency.ts`
- ☐ T071 [US3] 依 Specification FR-016 與 Plan 「Emergency + Live-Required 兩階段技術機制」，將 live search 與 emergencyGuide 結構化輸出整合至 `api/assistant.ts`，確保 Safety 與 Freshness 正交共存：Stage 1 回應不得等待 live search 完成，須先交付完整 `emergencyGuide` (immediateAction / nextAction / 可選 phrase / importantNotice)；Stage 2 (前端背景第二次 request) 補充 `liveDataStatus` / `liveDataMessage`，逾時或失敗時標記 `uncertain` / `unavailable`，且不得撤回或延後已顯示之 `emergencyGuide`；並以人工模擬 live lookup 延遲 / 失敗情境驗證 `emergency guidance` 仍先於 live 結果顯示
- ☐ T072 [P] [US3] 依 Specification FR-015 與 Plan 對應 technical contract，建立即時資訊狀態標籤元件於 `src/components/LiveDataStatus.tsx`

- ❑ T073 [P] [US3] 依 Specification FR-016 與 Plan 對應 technical contract，建立緊急指引高優先卡片元件於 `src/features/emergency/EmergencyAnswerCard.tsx`
- ❑ T074 [US3] 串接首頁「探索附近」、NearbyExplorer、即時狀態與 emergency 模式的互斥操作於 `src/App.tsx`

Run：啟動目前版本的附近探索、即時資料與緊急處理環境。

Verify：執行 US3 automated tests、build/type checks 與 Journeys 3–5 manual checks，確認 GPS / 手動 fallback、即時資訊誠實狀態與安全優先緊急指引。

Fix：若 Verify 失敗，只修正 Phase 5 直接相關的 source/test/config，然後重新 Run / Verify。

Evidence：記錄 commands、automated checks、manual journeys、pass/fail 與 deferred issues；不要求每個小 task 建立獨立報告。

Phase 6: Polish & Cross-Cutting Concerns

Technical Plan Traceability: Phase 1 App Shell & Foundation、Phase 9 Responsive Polish & Acceptance Criteria Verification

Purpose: 收斂 PWA、隱私、無障礙、響應式、更新流程與完整驗收。

- ❑ T075 建立 installable Web App manifest、standalone display 與必要 metadata 於 `public/manifest.webmanifest`
- ❑ T076 建立各支援尺寸的 Tokyo Mate PWA icons 於 `public/icons/`
- ❑ T077 實作 Service Worker module 於 `src/service-worker.ts`，先提供可被 registration 使用的版本化 App Shell 與靜態資產處理
- ❑ T078 依 Plan 之 PWA Architecture / Cache Boundary 技術決策，定義 Cache Storage allowlist 與 API exclusion 於 `src/service-worker.ts`
- ❑ T079 註冊已存在的 service worker module 於 `src/main.tsx`；不得在 T077/T078 完成前 import 或 registration
- ❑ T080 驗證 manifest、icons、secure context、standalone display 與支援平台安裝流程於 `specs/001-tokyo-travel-assistant/quickstart.md`
- ❑ T081 驗證首次線上載入後 offline 重新啟動的 App Shell 與批准靜態 Tokyo Knowledge Base 於 `specs/001-tokyo-travel-assistant/quickstart.md`
- ❑ T082 驗證 offline 嘗試 AI、Speech、Places、live-data 時顯示需要網路與重試，不出現無限 loading 或 raw network error，並驗證恢復網路後可重試於 `src/components/NetworkStatus.test.tsx`
- ❑ T083 驗證新版本提示、延後更新、安全時機套用與不強制中斷目前任務於 `src/components/UpdatePrompt.tsx`
- ❑ T084 依 FR-017 / FR-018 / FR-026 與 Plan 對應 privacy/cache technical design，驗證 LocalStorage、IndexedDB 與 Cache Storage 之 privacy boundary 於 `src/service-worker.test.ts`；驗證結果輸出至 `docs/verification/privacy-report.md` (Generated Verification Artifact，執行時產生)
- ❑ T085 [P] 依 Specification FR-025 之 acceptance baseline viewport matrix，完成手機 / 平板 / 桌面、直向 / 橫向、瀏覽器工具列與 safe-area 響應式樣式於 `src/styles/components.css`
- ❑ T086 [P] 驗證鍵盤焦點、語意標籤、觸控尺寸、動態狀態播報與 reduced-motion 於 `src/components/AccessibilityAudit.test.tsx`
- ❑ T087 執行並記錄驗收測試：(1) SC-001 操作步驟；(2) 依 Specification SC-002，執行 Plan 定義之 benchmark harness (讀取 `tests/fixtures/latency-cases.json`、於 client-to-render 邊界量測、輸出報告至 `docs/verification/latency-report.md`，web search/Places/STT/TTS 另行獨立量測)；(3) SC-003 一般旅遊 action-first 順序與最小契約；(4) SC-004 emergency 首屏 immediateAction/nextAction 優先順序；(5) 依 Specification FR-015 與 Plan 對應 technical contract，驗證 live-data 觸發邊界與最小契約；(6) FR-017 / SC-009 session retention 邊界驗證；(7) 依 Specification FR-020 與 Plan 的 QA implementation，執行台灣繁中詞彙 QA；(8) FR-025 / SC-014 viewport matrix 與 safe-area / 鍵盤 / 水平捲動 / touch target / standalone checks 於 `specs/001-tokyo-travel-assistant/quickstart-results.md`；並將本項目摘要輸出至 `docs/verification/journey-report.md` (latency 量測結果如上已輸出 `docs/verification/latency-report.md`，皆為 Generated Verification Artifact，執行時產生)
- ❑ T088 執行目前專案定義的完整 test、build 與 TypeScript/static checks；若失敗，定位實際受影響的 source/test/config，只修正直接相關檔案，不得刪除失敗測試、停用測試或降低驗收標準來取得通過；修正後重新執行完整驗證並留下驗證結果，並於全部驗收完成後彙整輸出 `docs/verification/final-report.md` (Generated Verification Artifact，執行時產生)
- ❑ T089 依 Journey 0–5 執行手機瀏覽器及 installed PWA 完整驗收，並在 FR-025 / SC-014 matrix 的每個 viewport 與情境驗證 top/bottom safe area、軟體鍵盤、主要翻譯內容無水平捲動、touch targets 不重疊及 standalone 操作，記錄結果於 `specs/001-tokyo-travel-assistant/quickstart-results.md`；PWA / viewport 相關結果並輸出至 `docs/verification/pwa-report.md` (Generated Verification Artifact，執行時產生)
- ❑ T090 [P] 撰寫安裝、環境變數、開發、測試、部署與隱私限制說明於 `README.md`

Run：啟動目前版本並執行 PWA、responsive、accessibility、privacy 與完整驗收所需環境。

Verify：執行 Phase 6 automated tests、build/type/static checks、PWA install/offline journeys、FR-025 / SC-014 matrix 與 quickstart acceptance checks，確認無秘密、精確位置、原始錄音或長期聊天資料被快取或持久化。

Fix : 若 Verify 失敗，只修正實際受影響且與 Phase 6 直接相關的 source/test/config，重新 Run / Verify；不得刪除失敗測試、停用測試或降低驗收標準。

Evidence : 留下 commands、automated checks、manual journeys、pass/fail、deferred issues 與最終驗證結果；不要求每個小 task 建立獨立報告。

Dependencies & Execution Order

Phase Dependencies

- **Phase 1 Setup**: 無依賴，可立即開始。
- **Phase 2 Foundational**: 依賴 Phase 1，並阻塞所有 User Story。
- **Phase 3 US1**: 依賴 Phase 2；是建議 MVP。
- **Phase 4 US2**: 依賴 Phase 2；可與 US1 平行開發，但整合 `api/assistant.ts` 與 `src/App.tsx` 時需依序合併。
- **Phase 5 US3**: 依賴 Phase 2；可與 US1 / US2 平行開發，但整合共用檔案時需依序合併。
- **Phase 6 Polish**: PWA implementation 與 registration 依序為 T075 → T079；T080 → T084 依序完成 installability、offline App Shell / static Knowledge Base、network-required fallback、update flow、cache privacy boundary。之後完成 T085 → T090；完整發布依賴 US1、US2、US3 全部完成。

User Story Dependencies

- **US1 (P1)**: Foundational 完成後即可開始，不依賴其他故事。
- **US2 (P1)**: Foundational 完成後即可開始；透過共用 Assistant contract 整合，但可用 mock provider 獨立測試。
- **US3 (P2)**: Foundational 完成後即可開始；Places 與 geolocation 可獨立完成，live/emergency 透過共用 Assistant contract 整合。

Within Each User Story

1. 先完成該故事的 tests，並確認在實作前會失敗。
2. 型別 / 靜態資料 / prompt 可平行完成。
3. services 與 provider adapters 完成後實作 endpoints。
4. UI components 完成後串接 screen 與 `src/App.tsx`。
5. 通過獨立測試與 checkpoint 後才視為故事完成。

Parallel Execution Examples

User Story 1

平行：T025 Assistant 翻譯測試、T026 Transcribe 測試、T027 Speech 測試、T028 UI 旅程測試
平行：T029 翻譯 prompt、T031 Transcribe endpoint、T032 Speech endpoint、T033 recorder service
平行：T034 VoiceInputModal、T035 EditableTranscript、T036 AudioPlayer、T037 TranslationResult
依序：T030 Assistant endpoint → T038 AssistantScreen → T039 HomeScreen → T040 App 整合

User Story 2

平行：T041 知識檢索測試、T042 Assistant travel 測試、T043 UI 旅程測試
平行：T044–T049 六類知識 JSON、T051 travel prompt、T053–T055 顯示元件
依序：T050 knowledge service → T052 Assistant 整合 → T056 KnowledgeBrowser → T057 App 整合

User Story 3

平行：T058–T062 Places、定位、即時資料、緊急模式與 UI 旅程測試
平行：T063 geolocation、T065 PlaceCard、T066 NearbyResults、T068 live-search、T069–T070 prompts、T072–T073 狀態元件
依序：T064 Places endpoint → T067 NearbyExplorer；T068–T070 → T071 Assistant 整合 → T074 App 整合

依序：T075 manifest → T076 icons → T077 service worker implementation → T078 cache allowlist/API exclusion → T079 registration

依序：T080 installability → T081 offline App Shell/static Knowledge Base → T082 network-required fallback → T083 update flow → T084 cache privacy boundary

平行：T085 responsive styles、T086 accessibility audit

依序：T087 SC-001/002/004 與 FR-015/020 verification → T088 test/build/typecheck → T089 Journey 0-5 acceptance

Implementation Strategy

MVP First (只交付 User Story 1)

1. 完成 Phase 1 Setup。
2. 完成 Phase 2 Foundational。
3. 完成 Phase 3 US1。
4. 停止擴充並獨立執行 Journey 1、US1 tests 與 build。
5. 通過後即可部署翻譯與口譯 MVP。

Incremental Delivery

1. Setup + Foundational → 可實作基礎。
2. US1 → 雙向即時翻譯 MVP。
3. US2 → 東京知識與 action-first 私人導遊。
4. US3 → 附近探索、即時資訊與緊急模式。
5. Polish → 依序完成 PWA implementation、cache boundary、registration、installability/offline/fallback/update/privacy verification，再完成 responsive、完整驗收與發布。

Parallel Team Strategy

1. 團隊共同完成 Setup 與 Foundational。
2. Developer A 負責 US1，Developer B 負責 US2，Developer C 負責 US3。
3. 各故事先在獨立測試與 mock provider 下完成。
4. `api/assistant.ts` 與 `src/App.tsx` 的故事整合依 US1 → US2 → US3 順序合併並重跑相關測試。

Notes

- [P] 僅標示不同檔案且沒有未完成前置依賴的工作；PWA T075–T084 的順序性任務不標示 [P]。
- [US1]、[US2]、[US3] 提供需求追溯；Setup、Foundational、Polish 不加 Story 標籤。
- 不加入帳號、資料庫、向量庫、長期聊天歷史、背景定位或背景錄音。
- API keys 僅能由 `api/` server boundary 讀取，不得使用 `VITE_` 前綴或進入 frontend bundle。
- 每個 checkpoint 都遵循 Run → Verify → Fix，驗證失敗不得直接進入下一個主要階段。